ConsoleUI Refresher Notes

UBITE (UCC BITE) | Owner: Bolivar | Contract Section 8.5

1. The big idea

`ConsoleUI` is the **only** class that prints to the screen or reads the keyboard. It formats output and collects *validated* input. It makes **no decisions** and does **no business math**. Main decides what happens; ConsoleUI just shows it and asks.

Must not: do price math, touch the queue, read/write files, decide program flow, or call `System.exit`.

2. Fields and constants

Name	What it is
`getInput`	The ONE `Scanner` on `System.in` for the whole program
`CURRENCY`	`"PHP "` (with a trailing space on purpose), put in front of every money value

3. Private helpers (only ConsoleUI can use them)

Method	What it does
`heightSpaces(int count)`	Prints `count` blank lines (vertical spacing)
`nextLineOrThrow()`	Reads one line. If input has ended (Ctrl+Z), converts `NoSuchElementException` into `IllegalStateException("Input closed")` so Main can shut down cleanly
`formatMoney(double)`	`CURRENCY + String.format("%.2f", amount)`, so `65` becomes `PHP 65.00`. Formats only, no math

Method	What it does
`shorten(text, max)`	`null` becomes `""`; short text is returned as is; long text is cut to `max - 3` characters plus `...`

4. Public methods at a glance

Method	Reads input?	Quick summary
`showBanner()`	Enter only	ASCII art + subtitle, then waits for Enter
`showMainMenu()`	No	Prints options 1 to 6 only
`readInt(prompt, min, max)`	Yes	Loops until a whole number in range
`readLine(prompt)`	Yes	Returns the trimmed line (may be empty)
`confirm(prompt)`	Yes	Loops until y/yes/n/no, returns true/false
`pause()`	Enter only	"Press Enter to continue..."
`printMenu(MenuItem[])`	No	Table of id, name, price, SOLD OUT
`printOrders(title, Order[])`	No	Titled table of orders
`printReceipt(order, queuePosition)`	No	Receipt after placing an order
`showInfo / showSuccess / showWarning / showError`	No	Print with `[INFO]`, `[OK]`, `[WARNING]`, `[ERROR]`
`showGoodbye()`	No	Closing message
`ClearCMDScreen()`	No	Clears the console with an ANSI code (currently unused)
`close()`	No	Closes the Scanner. Call last, once

5. The input methods, step by step

readInt

```
while (true) {
    System.out.print(prompt);
    String line = nextLineOrThrow().trim();
    int value;
    try { value = Integer.parseInt(line); }
    catch (NumberFormatException e) { print msg; continue; }
    if (value < min || value > max) { print msg; continue; }
    return value;
}
```

1. If `min > max`, throw `IllegalArgumentException` (a programmer mistake, checked first).
2. Read a whole line and `trim()` it.
3. Not a number: "Please enter a whole number.", then `continue`.
4. Out of range: "Please enter a number from MIN to MAX.", then `continue`.
5. Valid: `return value`. This is the only way out of the loop.

confirm

- Prints `prompt + " (y/n): "`.
- `trim().toLowerCase()` first, so `" YES "` works.
- `y` / `yes` returns `true`; `n` / `no` returns `false`; anything else prints "Please type y or n." and loops.

readLine, pause

- `readLine`: print prompt, return `nextLineOrThrow().trim()`.
- `pause`: print the text, call `nextLineOrThrow()` and ignore the result. It just waits for Enter.

6. Display logic

printMenu

- Header line, then if `menu == null || menu.length == 0` print "The menu is empty." and `return`.
- Format `"%-5s %-22s %12s %s%n"` gives No. (5, left), Item (22, left), Price (12, right), then the status text.
- Print the header row, then a dashed line, then one row per item, then a dashed line.
- Status is `""` or `"SOLD OUT"` (ternary).
- The number shown is `item.getId()`, which is what the customer types.

printOrders

- Prints `=== title ===`. Empty or null prints "(no orders to show)" and returns.
- Format `"%-10s %-22s %4s %12s %12s %s%n"` gives Order No., Item, Qty, Unit Price, Total, Status.
- Order number is `String.format("#%04d", id)`, so 7 becomes `#0007` (display only).
- Item names go through `shorten(..., 22)`; money goes through `formatMoney`.
- Every value comes from Order getters. **Nothing is calculated here.**

printReceipt

- Plain `println` lines with labels padded by hand (`"Order No. : "`), the `#0007` style number, formatted money, and "Your position in the queue: N".

Message methods

Each one is just `System.out.println("[TAG] " + message)`. Main picks which one to call (error for sold out, warning for skipped CSV rows, and so on).

7. New syntax cheat sheet (the stuff that is easy to forget)

Enhanced for loop (for-each)

```
for (MenuItem item : menu) { ... }
```

Read `:` as "in". `item` is each element of `menu` in turn. Use it when you do not need the index. Use the classic `for (int i = 0; ...)` when you need `i`.

Ternary operator

```
String status = item.isAvailable() ? "" : "SOLD OUT";
```

`condition ? valueIfTrue : valueIfFalse`. A one-line if/else that gives back a value.

continue

Skips the rest of this pass of the loop and jumps back to the top of the `while`. In `readInt`'s `catch`, it also guarantees `value` is assigned before it is used, which the compiler needs.

trim()

Removes spaces at the start and end only. `Integer.parseInt(" 5")` fails but `"5"` works. Spaces in the middle stay.

printf / String.format placeholders

Placeholder	Meaning
`%s`	a String, as is
`%10s`	String, at least 10 wide, **right**-aligned
`%-10s`	String, at least 10 wide, **left**-aligned (the `-` flips it)
`%.2f`	decimal with exactly 2 digits after the dot
`%04d`	whole number padded with zeros to 4 digits (`7` becomes `0007`)
`%n`	a portable line break

- Width is a **minimum**. Longer text is not cut, which is why `shorten` exists.
- Plain spaces between placeholders are printed as is (they make the column gaps).
- One `format` string is reused for the header and every row, so columns line up.

nextLine vs nextInt

Always use `nextLine()`. `nextInt()` leaves the Enter key behind in the buffer, and the next `nextLine()` then reads an empty line by mistake. Reading whole lines and parsing them ourselves avoids this.

try / catch

`Integer.parseInt` throws `NumberFormatException` on bad text. We catch it and turn it into a friendly message instead of crashing.

Exceptions used on purpose

Exception	Where	Meaning
`IllegalArgumentException`	`readInt` when `min > max`	Programmer error
`IllegalStateException("Input closed")`	`nextLineOrThrow`	Input stream ended; Main catches it at the top and shuts down cleanly

Escape characters in strings

- `\\` prints one backslash (the ASCII banner uses it a lot).
- `\033[H\033[2J` is an ANSI code (move cursor home, clear screen). It may not work in old Windows consoles.

while (true)

An "infinite" loop that only ends through `return` (or a thrown exception). `readInt` and `confirm` use it to keep asking until the answer is valid.

8. Quick self-check

- Why only one Scanner? Several Scanners on `System.in` can swallow each other's input.
- Why does ConsoleUI never calculate a total? Order does it once; ConsoleUI just displays `getTotal()`.

- Why `nextLineOrThrow()` instead of calling `nextLine()` directly? One place converts end-of-input into the exception the contract requires.
- Why `shorten` before printing a name? Width is a minimum, so an overly long name would push the columns out of line.